# Borg IA Cube: a distributed CNN system with a multiplatform chat and RabbitMQ middleware

Espinoza Benavente Hector
Inga Chávez Jorly
Villanueva Alzamora Farid

Universidad Nacional de Ingeniería – Facultad de Ciencias

CC4P1 – Concurrent and Distributed Programming
Practical Assessment 3 – 2026-II

## Contents

**1 Introduction** **2**

**2 Theoretical framework** **2**
2.1 Convolutional neural networks . . . 2
2.2 Messaging middleware: RabbitMQ . . . 2
2.3 Distributed systems and sockets . . . 3
2.4 Concurrency and synchronization . . . 3
2.5 Database transactions . . . 3
2.6 Performance . . . 3

**3 Methodology and implementation** **3**
3.1 System architecture . . . 3
3.2 AI models and micro chunks . . . 5
3.3 Communication protocol . . . 6
3.4 Heterogeneous databases . . . 7
3.5 Concurrency concepts in the code . . . 8
3.6 Deployment . . . 8
3.7 Experimental design . . . 9

**4 Results and discussion** **10**
4.1 Model metrics . . . 10
4.2 Training in the 4 configurations . . . 11
4.3 Inference in the 4 configurations . . . 12
4.4 Saturation test . . . 13
4.5 1000 registrations and database validation . . . 14
4.6 General discussion . . . 14

**5 Conclusions** **14**

**6 Source code appendix** **15**

**7 Documentation appendix** **15**

# 1 Introduction

*Borg IA Cube* continues the *Open AI Cubes* concept from the midterm exam: a modular system in which each "cube" provides one artificial intelligence capability. In this assignment the user sends a photo of a cell through a **chat** (Android, desktop or web) and receives the diagnosis of a convolutional neural network (CNN). There are two cubes: **model A** detects malaria (parasitized or uninfected cell) and **model B** classifies 8 types of blood cells.

Unlike the midterm, the nodes no longer talk to each other directly. Clients connect to the *backend* with **custom TCP sockets**, and the backend, the AI nodes and the database nodes communicate only through **RabbitMQ**, which acts as an asynchronous messaging *middleware*. The system uses **three databases with different engines** (PostgreSQL, MariaDB and MySQL): two store the results of each model and the third validates clients, products and images.

The CNNs are written from scratch in Go, without TensorFlow or PyTorch. The system was deployed with the server on **Google Cloud** and one laptop per team member, each with its database and its AI node, connected over the internet from three different homes.

The goals of this work are:

- Implement a CNN with convolution and MLP *micro chunks*, trained on datasets chosen by the team, in the sequential, parallel, distributed and parallel-distributed configurations.
- Integrate Android, desktop and web clients, a *backend*, AI nodes and three heterogeneous databases through custom sockets and RabbitMQ.
- Keep the databases consistent under concurrent writes.
- Evaluate performance and saturate the system with 1000 transactions per client.

# 2 Theoretical framework

## 2.1 Convolutional neural networks

A CNN is a deep learning architecture designed for grid-structured data such as images. It has a convolutional part, which extracts features, and a multilayer perceptron (MLP), which classifies the input from those features.

### 2.1.1 Convolution, ReLU and pooling

A convolution slides small filters (here $3 \times 3$) over the image. Each filter produces an activation map that responds to a local pattern, such as an edge or a texture:

$$\text{out}[f][y][x] = b_f + \sum_{c} \sum_{k_y} \sum_{k_x} W_{f,c,k_y,k_x} \, \text{in}[c][y + k_y - 1][x + k_x - 1].$$

The ReLU function, $\max(0, z)$, introduces non-linearity, and $2 \times 2$ *max pooling* halves each spatial dimension while keeping the maximum value of each window.

### 2.1.2 Multilayer perceptron and softmax

The final map is flattened into a vector that goes through fully connected layers. The last one produces one value per class, which the softmax function turns into probabilities. Training minimizes the cross-entropy loss with stochastic gradient descent (SGD) with momentum.

## 2.2 Messaging middleware: RabbitMQ

RabbitMQ is a message broker that implements the AMQP protocol. Producers publish messages to an *exchange*, which routes them to queues; consumers read from those queues at their own pace. This decouples the stages: if a consumer is slow, messages wait in the queue instead of being lost. Three RabbitMQ features are key in this system:

- **Load sharing among consumers:** if several nodes consume the same queue, RabbitMQ distributes the messages among them, so AI nodes can be added without changing anything else.
- **Prefetch and acknowledgement (*ack*):** each consumer receives at most *prefetch* unacknowledged messages at a time. An unacknowledged message is redelivered if the consumer fails.
- **RPC:** the caller publishes with a reply queue (`reply_to`) and an identifier (`correlation_id`), and the callee publishes its answer to that queue with the same identifier.

### 2.3 Distributed systems and sockets

A distributed system is made of autonomous processes that communicate over a network to work on a common task. Communication with the clients uses TCP *sockets*: each socket is one endpoint of a bidirectional channel between two processes, and TCP guarantees reliable, in-order delivery.

### 2.4 Concurrency and synchronization

When several threads access shared data without coordination, a *race condition* occurs: the result depends on the non-deterministic order in which the operating system interleaves the threads. The tools used to avoid it are:

- **Mutual exclusion:** `ReentrantLock`, `synchronized` and `sync.Mutex`.
- **Concurrent structures:** `ConcurrentHashMap`, `LinkedBlockingQueue` and atomic counters.
- **Producer–consumer:** blocking queues and Go channels.
- **Barriers:** a group of tasks waits until all of them finish before continuing (`sync.WaitGroup`).
- **Thread-local state:** each thread works on its own copy of the data (for example, its own gradient buffer), and the copies are combined at the end.

### 2.5 Database transactions

A transaction groups several operations that are applied all together or not at all (atomicity). To prevent two concurrent transactions from reading the same value and overwriting each other, `SELECT ... FOR UPDATE` locks the row until the transaction ends. An **idempotent** insert (`ON CONFLICT DO NOTHING` in PostgreSQL, `INSERT IGNORE` in MySQL and MariaDB) does not duplicate the record if the same message arrives twice.

### 2.6 Performance

The metrics used are:

- **Latency:** time between a client sending a query and receiving the answer.
- **Throughput:** transactions processed per second.
- **Speedup:** $S = T_{\text{sequential}}/T$.
- **Parallelization cost:** the time that is not useful computation (sending data, waiting and combining results). It is what keeps $S$ from growing linearly.

## 3 Methodology and implementation

### 3.1 System architecture

Figure 1 shows the architecture. Clients and nodes always open the connection to the server: clients to the backend (port 5000) or to the web server (port 8080), and nodes to RabbitMQ (port 5672). Therefore no ports had to be opened in the team members' homes.

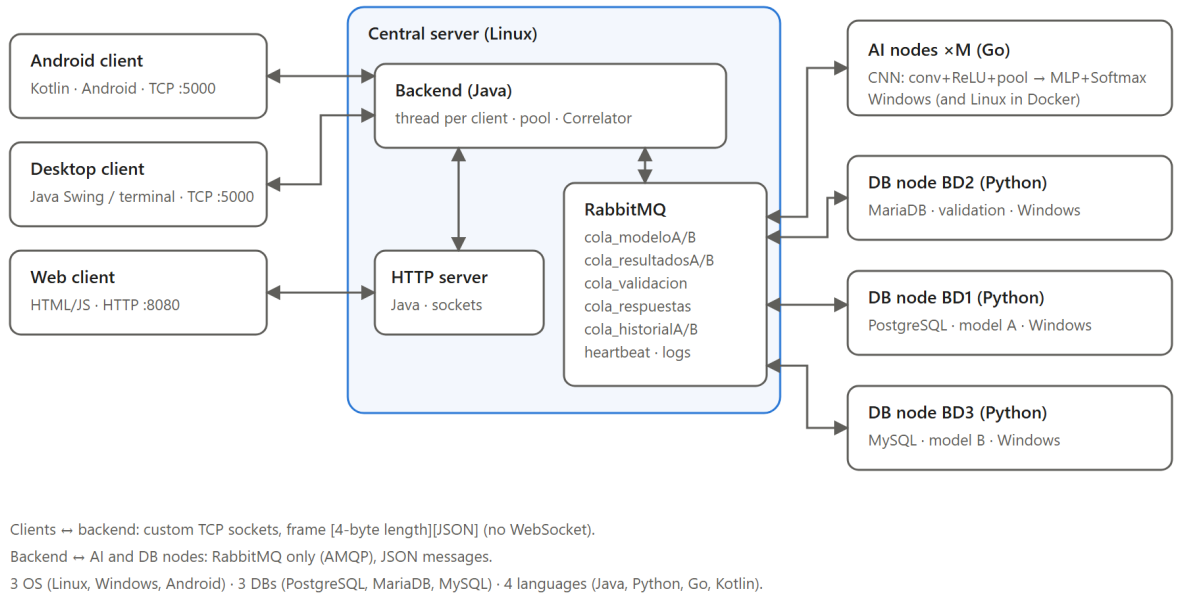



Figure 1: Borg IA Cube architecture: clients, central server with RabbitMQ, AI nodes and database nodes.

### 3.1.1 Modules

- **Clients:** an Android app in Kotlin (Figure 2), a Java Swing desktop chat with a terminal version, and an HTML/JS web chat. All three can log in, register, send an image, view the history and launch a stress test.
- **Web server:** a custom HTTP server built with Java's `ServerSocket`. It serves the web chat and forwards every browser action to the backend over TCP. It does not use WebSocket.
- **Backend:** receives the three clients, validates users against BD2, routes the images to RabbitMQ and sends each answer back to the right client.
- **AI nodes:** run the CNN on the images of `cola_modeloA` and `cola_modeloB`, and serve the distributed training tasks.
- **DB nodes:** BD2 validates; BD1 and BD3 store the results of models A and B.

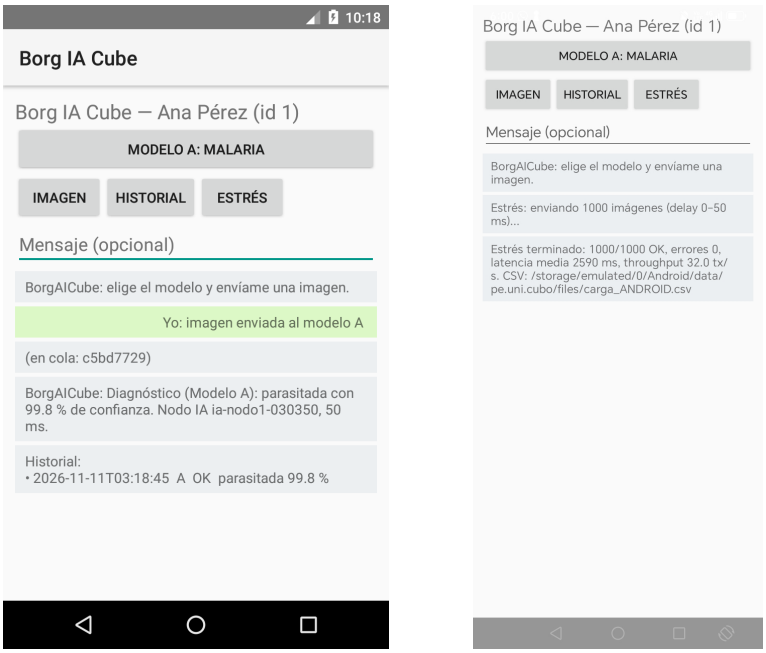



Figure 2: Android app (Spanish interface). Left: diagnosis of a parasitized cell (99.8 %) and history. Right: stress test of 1000 images against the server on Google Cloud, with no errors.

### 3.1.2 Code components

| Component | Responsibility |
|---|---|
| `java/src/cubo/backend` | `Backend`, `ManejadorCliente`, `PuenteRabbit` (topology, publisher and RPC), `Correlador` and `Sesion`. |
| `java/src/cubo/web` | Custom HTTP server (`ServidorHttp`, `PeticionHttp`, `RespuestaHttp`). |
| `java/src/cubo/desktop` | Swing chat (`ClienteDesktop`) and terminal chat (`ClienteTerminal`). |
| `java/src/cubo/carga` | `GeneradorCarga`: queries or registrations with a random *delay*. |
| `go/cnn` | CNN, backpropagation, dataset, preprocessing and parallel training. |
| `go/distrib, go/rmq` | Distributed training over RabbitMQ; topology, heartbeat and metrics. |
| `go/cmd/nodo`, `go/cmd/entrenar` | AI node and training in the 4 configurations. |
| `python/` | `nodo_bd.py` (BD1, BD2, BD3), `validador.py`, `monitor.py`, `verificar_consistencia.py`. |
| `android/` | `Conexion.kt` (socket and protocol) and `MainActivity.kt`. |
| `sql/` | Creation scripts for BD1, BD2 and BD3. |

Table 1: Main components of the implementation.

## 3.2 AI models and micro chunks

### 3.2.1 Datasets

| | Model A: malaria | Model B: blood cells |
|---|---|---|
| Source | NIH, *Malaria Cell Images* [2] | BloodMNIST, from MedMNIST v2 [3], derived from [4] |
| Images | 27 558 (13 779 per class) | 17 092 |
| Classes | parasitized, uninfected | basophil, eosinophil, erythroblast, immature granulocyte, lymphocyte, monocyte, neutrophil, platelet |
| Split | 22 046 / 2 754 / 2 758 (80/10/10, stratified) | 11 959 / 1 712 / 3 421 (official split) |
| Preprocessing | Black padding to a square, bilinear resizing to $32 \times 32$, RGB | $28 \times 28$ RGB images |

Table 2: Datasets chosen by the team (training / validation / test).

The datasets are stored in a custom binary format: a `DSET` header with $N$, $C$, $H$, $W$ and $K$, and for each image `[split][label][C×H×W bytes]`. Each channel is normalized with the mean and standard deviation of the training set. The AI node applies the same padding, resizing and normalization to the photos that arrive through the chat.

### 3.2.2 Network architecture

| Micro chunk | Layer | Input (A) | Output (A) | Detail |
|---|---|---|---|---|
| Convolutions | conv1 + ReLU | $3 \times 32 \times 32$ | $8 \times 32 \times 32$ | $3 \times 3$ filter, padding 1 |
| | pool1 | $8 \times 32 \times 32$ | $8 \times 16 \times 16$ | $2 \times 2$ max |
| | conv2 + ReLU | $8 \times 16 \times 16$ | $16 \times 16 \times 16$ | $3 \times 3$ filter, padding 1 |
| | pool2 | $16 \times 16 \times 16$ | $16 \times 8 \times 8$ | $2 \times 2$ max |
| MLP | fc1 + ReLU | 1 024 | 64 | fully connected |
| | fc2 + softmax | 64 | $K$ | one output per class |

Table 3: CNN layers: 67 122 weights in model A and 52 152 in model B.

Backpropagation was written by hand. An automated test (`go test ./cnn`) compares the analytical gradient with the numerical one on random weights.

### 3.2.3 Parallel training: one buffer per thread and one barrier per batch

Each batch of 32 images is split among $H$ goroutines. Each one accumulates the gradient in *its own* buffer, so there are no shared writes and no race condition. A `sync.WaitGroup` acts as the barrier, and then the buffers are added and the weights are updated:

```
func (e *Entrenador) Paso(d *Dataset, lote []int) (perdida float64, aciertos int) {
    var wg sync.WaitGroup
    for h := 0; h < e.Hilos; h++ {
        wg.Add(1)
        go func(h int) {
            defer wg.Done()
            g := e.grads[h]                              // the goroutine's own buffer
            for j := h; j < len(lote); j += e.Hilos {  // interleaved split of the batch
                e.Red.Muestra(d, lote[j], e.xs[h])
                e.Red.Gradiente(e.trabajos[h], e.xs[h], int(d.Etiq[lote[j]]), g)
            }
        }(h)
    }
    wg.Wait()   // barrier: every goroutine finished its share of the batch
    // the H buffers are added and SGD with momentum is applied
    ...
}
```

### 3.2.4 Distributed training: model averaging over RabbitMQ

The coordinator (`entrenar -modo dist`) publishes one task per node to `cola_entrenamiento`, with the current weights and the images assigned to that node. Each AI node runs 10 SGD steps on its share, with 1 thread (*dist*) or with all its cores (*pardist*), and returns its weights to the coordinator's private queue. The coordinator averages them, weighted by the number of images, and starts the next round.

## 3.3 Communication protocol

### 3.3.1 Clients and backend: custom TCP sockets

Each message is a frame `[4 bytes with the length N, big-endian][N bytes of UTF-8 JSON]`. The JSON is built and parsed by hand. A query carries `idCliente`, `tipoConsulta`, the image in Base64 (`payload`) and `timestamp`:

```
-> {"tipo":"CONSULTA","token":"9f1c...","idCliente":1,"tipoConsulta":"IMAGEN","modelo":"A",
    "formato":"png","payload":"iVBORw0KGgo...","timestamp":1794367427500}
<- {"tipo":"ACK","idSolicitud":"5e0b...","estado":"EN_COLA"}
<- {"tipo":"RESULTADO","idSolicitud":"5e0b...","etiqueta":"parasitada","probabilidad
    ":0.998,
    "nodoIA":"ia-integrante02-...","latenciaTotalMs":1404}
```

### 3.3.2 Backend and nodes: RabbitMQ queues

| Queue | Consumer | Content |
|---|---|---|
| `cola_modeloA / B` | AI nodes | `idSolicitud`, image, `modeloDestino`, `timestamp` |
| `cola_resultadosA / B` | BD1 / BD3 | The above plus label, probabilities and AI node |
| `cola_validacion` | BD2 | RPC validations: login, registration, request and image |
| `cola_respuestas` | Backend | Final result of each `idSolicitud` |
| `cola_historialA / B` | BD1 / BD3 | User history queries (RPC) |
| `cola_entrenamiento` | AI nodes | Distributed training tasks |
| `cola_heartbeat, cola_logs, cola_metricas` | Backend, monitor | Control: live nodes, logs, CPU and memory |

Table 4: RabbitMQ queues (exchanges `ex_cubo` and `ex_control`).

### 3.3.3 Protocol diagram

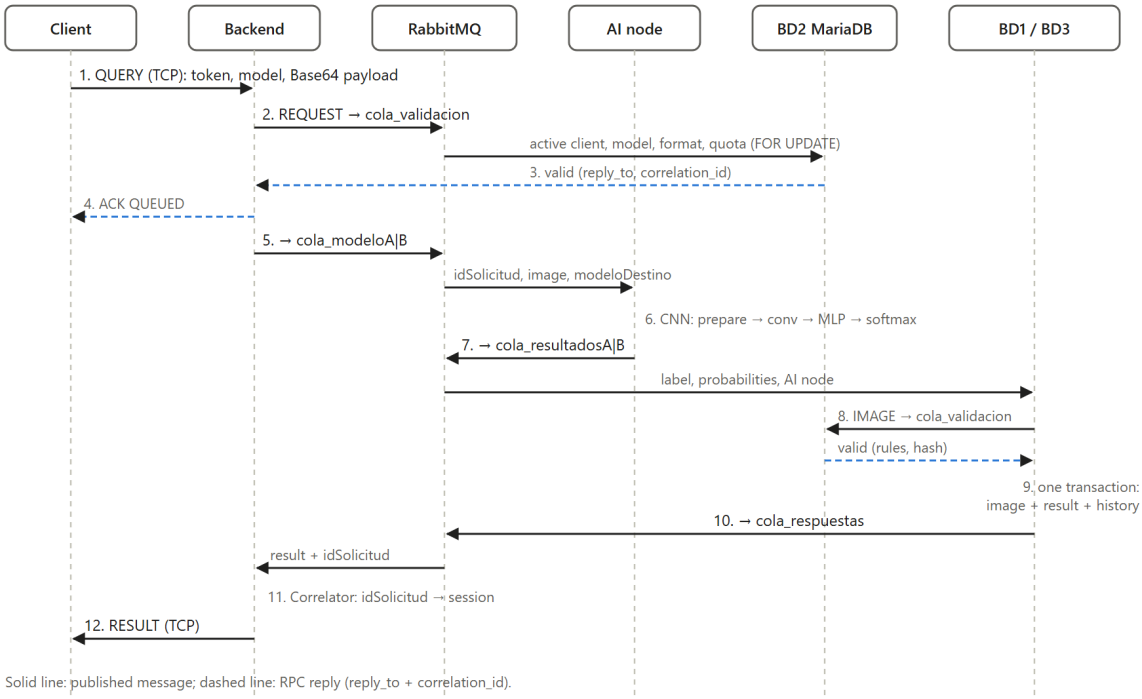



Figure 3: Flow of one query: Client → Backend → RabbitMQ → AI node → RabbitMQ → DB node → RabbitMQ → Backend → Client.

BD2 validates each query twice: before the AI (active client, enabled model, format and quota) and before storing (image rules and blocked *hash*). The backend uses the `Correlador` to know which socket each result goes to. Its `remove` on a `ConcurrentHashMap` is atomic: if the answer and the *timeout* arrive at the same time, only one of them "wins" and the client receives a single message.

## 3.4 Heterogeneous databases

| DB | Engine | Tables | Role |
|---|---|---|---|
| BD1 | PostgreSQL 16 | `imagenes`, `resultados`, `historial_consultas`, `metricas_entrenamiento` | Model A: metadata, label, probability, date and history |
| BD2 | MariaDB 11 | `clientes`, `modelos`, `cliente_modelo`, `reglas_imagen`, `imagenes_bloqueadas`, `auditoria_validaciones` | Validation of the client, the product (model) and the image |
| BD3 | MySQL 8 | Same as BD1 | Model B |

Table 5: Databases. The creation scripts are in `sql/`.

Each DB node handles messages with a thread *pool* and **one connection per thread**, never shared. Image, result and history are stored in a single transaction, and the client's daily quota is consumed with `FOR UPDATE`:

```
def _solicitud(cur, msg):
    """Validates active client, enabled model, format/size and consumes quota (with FOR
        UPDATE)."""
    cur.execute("SELECT estado, cuota_diaria, usadas_hoy, fecha_cuota < CURDATE() FROM
        clientes "
                "WHERE id_cliente = %s FOR UPDATE", (id_cliente,))
    ...
    cur.execute("UPDATE clientes SET usadas_hoy = %s, fecha_cuota = CURDATE() WHERE
        id_cliente = %s",
                (usadas + 1, id_cliente))
```

| **Concurrency test** (`pruebas_concurrencia.py`) | **Result** |
|---|---|
| 20 threads insert the same `id_solicitud` into BD1 and BD3 | 1 row per table |
| 50 threads request validation for a client with a quota of 10 in BD2 | exactly 10 approved |
| 20 threads register the same new user in BD2 | exactly 1 created |

Table 6: Concurrent write tests on the real databases.

## 3.5 Concurrency concepts in the code

| **Component** | **Threads and synchronization** |
|---|---|
| Backend (Java) | One thread per connection (`CachedThreadPool`) and a fixed *pool* for queries. Each session writes to its socket with a `ReentrantLock`, so frames never interleave. `ConcurrentHashMap` for sessions and pending requests; publications go through a `LinkedBlockingQueue` drained by a single publisher thread. Counters with `AtomicLong`. |
| HTTP server (Java) | *Pool* of 128 threads; `ConcurrentHashMap` of web sessions. |
| AI node (Go) | Producer–consumer: RabbitMQ delivers up to $H$ messages (*prefetch*) and $H$ goroutines read them from a Go channel. Publisher protected with `sync.Mutex`; `atomic.Int64` counters. |
| DB nodes (Python) | `ThreadPoolExecutor`, one connection per thread, transactions, `FOR UPDATE`, idempotent inserts and `threading.Lock` for the counters. |
| Android (Kotlin) | Reader thread, *ping* thread and socket writes inside `synchronized`; the UI is only touched from `runOnUiThread`. |

Table 7: Concurrency in each component.

## 3.6 Deployment

### 3.6.1 Team cluster

| | **Member01 (Farid)** | **Member02 (Hector)** | **Member03 (Jorly)** |
|---|---|---|---|
| Database | BD2 – MariaDB | BD1 – PostgreSQL | BD3 – MySQL |
| Nodes | BD2 (Python) + AI (Go) | BD1 (Python) + AI (Go) + load | BD3 (Python) + AI (Go) |
| Processor | Intel Core i7-1255U | Intel Core i5-13420H | AMD Ryzen 5 5500U |
| Cores / threads | 10 / 12 | 8 / 12 | 6 / 12 |
| RAM | 16 GB | 32 GB | 16 GB |
| Operating system | Windows 11 | Windows 11 (DB and AI in Docker) | Windows |

Table 8: Cluster laptops, each connected from a different home network.

The mobile client runs on an Android phone. The system therefore uses **3 operating systems** (Linux on the server, Windows on the laptops and Android on the phone), **3 database engines** on 3 independent nodes and **4 languages**: Java on the server, Python on the DB nodes, Go on the AI nodes and Kotlin on the mobile client. On Member02's laptop, Windows does not allow unsigned binaries to run, so its database and its AI node run in Docker containers.

### 3.6.2 Server on Google Cloud

The server runs on the same virtual machine as in the midterm: `oac-servidor`, an `e2-micro` instance with Ubuntu 24.04 LTS in the `southamerica-west1` region (Santiago, Chile), with a static IP. It runs RabbitMQ, the backend and the web server. A firewall rule opens TCP ports 5000, 8080 and 5672. The machine is prepared over SSH with `scripts/nube_preparar_vm.sh`, which installs RabbitMQ and Java, creates the RabbitMQ user with a random password and compiles the project. The backend and the web server run inside `tmux`, so they keep running after the SSH session is closed (Figures 4 and 5).

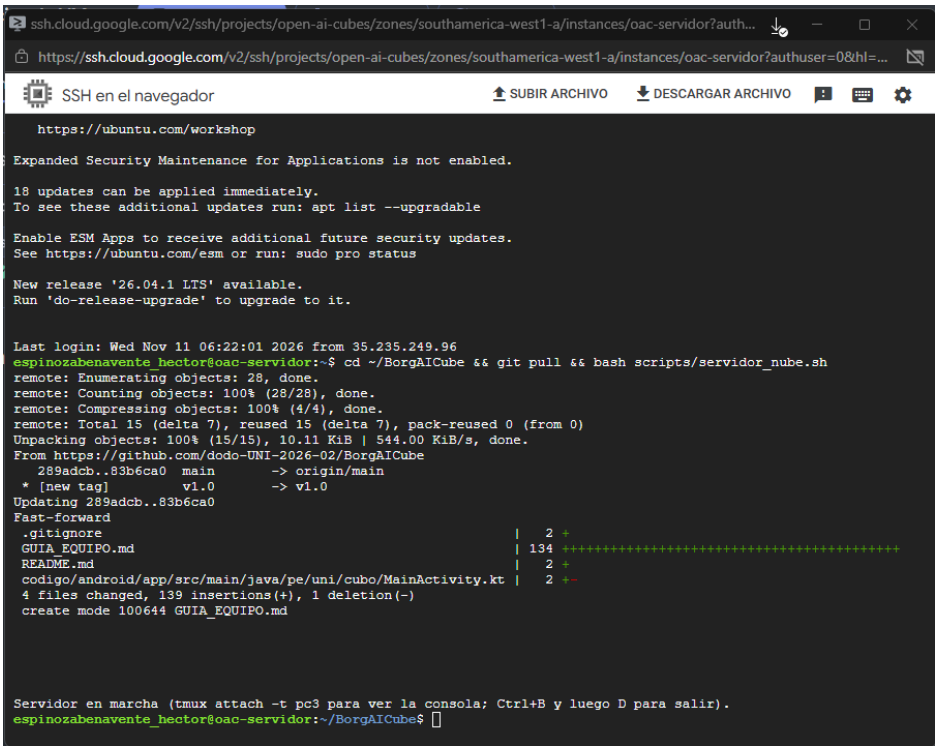



Figure 4: Starting the server on the Google Cloud VM over SSH from the browser.

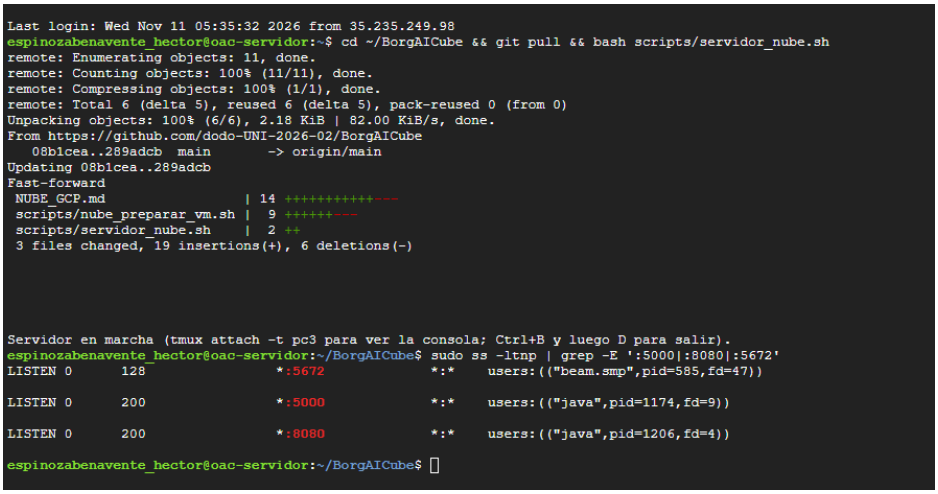



Figure 5: Processes listening on the VM: RabbitMQ (5672), backend (5000) and web server (8080).

## 3.7 Experimental design

The measurements were made on 2026-11-11 with the real cluster. The load and the training coordinator were launched from Member02's laptop. To change configuration, only the AI nodes were restarted with different parameters:

| Configuration | AI nodes | Experiments |
|---|---|---|
| Sequential | 1 node, 1 core and 1 thread | Inference; training of A and B |
| Parallel | 1 node, 12 threads | Inference; training of A and B |
| Distributed | 3 laptops, 1 thread each | Inference; training of A and B |
| Parallel distributed | 3 laptops, 12 threads each | Inference; training of A and B; saturation; 1000 registrations |

Table 9: Measured configurations.

- **Training:** 3 epochs, batch size 32, SGD with $\eta = 0.01$ and momentum 0.9, the same seed in every run.
- **Inference:** 1000 queries from the desktop client and 1000 from the web client at the same time, with 32 sessions and a random *delay* of 0 to 50 ms between sends.
- **Saturation:** desktop, web and Android, 1000 transactions each, at the same time.
- **Instrumentation:** `monitor.py` records every 0.5 s the depth of the RabbitMQ queues and the CPU and memory reported by each node. Everything is stored in CSV files, from which the charts are generated.

# 4 Results and discussion

## 4.1 Model metrics

The delivered models were trained for 12 epochs in parallel mode.

| Model | Training acc. | Validation acc. | Test acc. |
|---|---|---|---|
| A (malaria) | 97.3 % | 96.6 % | 95.2 % |
| B (blood cells) | 94.4 % | 90.0 % | 88.6 % |

Table 10: Accuracy of the delivered models.

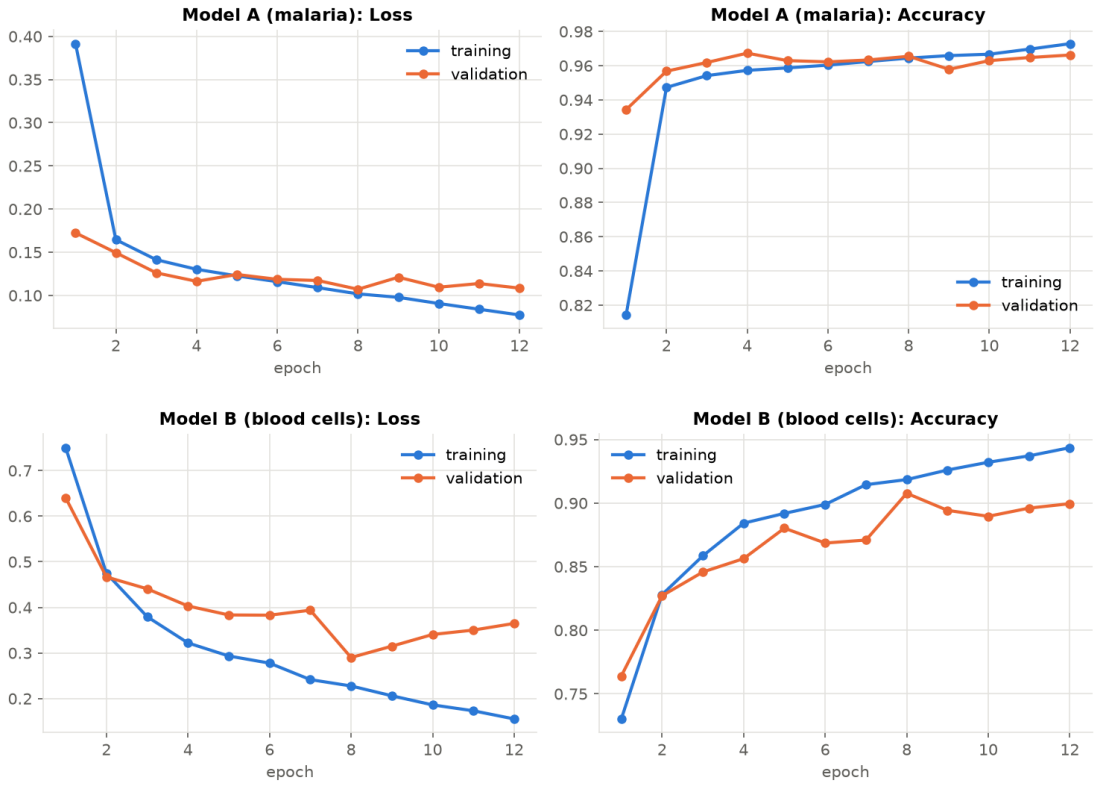



Figure 6: Loss and accuracy per epoch of models A and B.

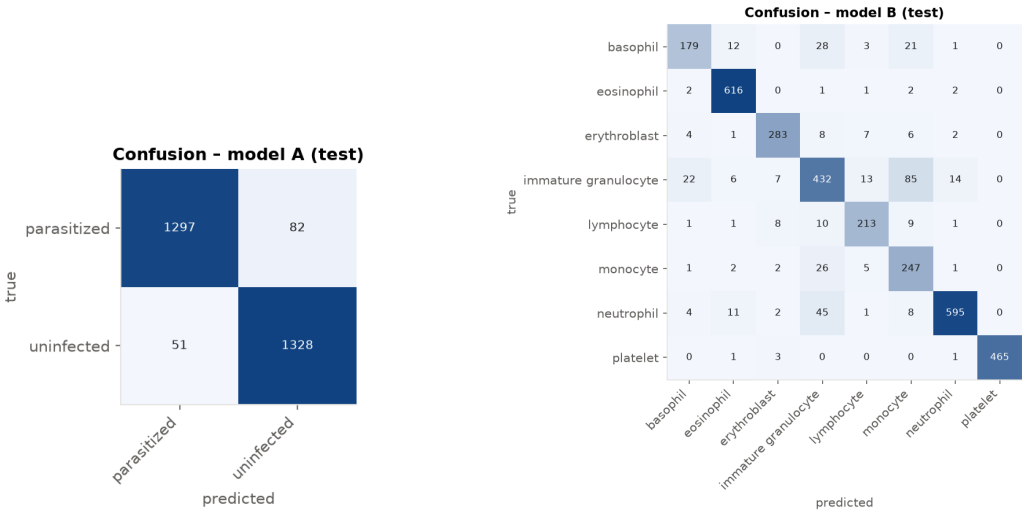



Figure 7: Confusion matrices on the test set.

In model B, most errors confuse immature granulocytes with monocytes and neutrophils, which are morphologically similar. Platelets and eosinophils are almost never confused.

## 4.2 Training in the 4 configurations

| | | Model A | | | Model B | | |
|---|---|---|---|---|---|---|---|
| **Configuration** | **Mach. × threads** | **s/epoch** | **Speedup** | **Acc.** | **s/epoch** | **Speedup** | **Acc.** |
| Sequential | 1 × 1 | 38.4 | 1.00 | 95.3 % | 16.6 | 1.00 | 83.7 % |
| Parallel | 1 × 12 | 6.4 | 6.00 | 95.0 % | 2.8 | 5.88 | 83.9 % |
| Distributed | 3 × 1 | 56.3 | 0.68 | 82.3 % | 26.3 | 0.63 | 80.2 % |
| Parallel distributed | 3 × 12 | 30.1 | 1.27 | 83.4 % | 16.6 | 1.00 | 80.6 % |

Table 11: Training on the real cluster (3 epochs). Speedup relative to sequential.

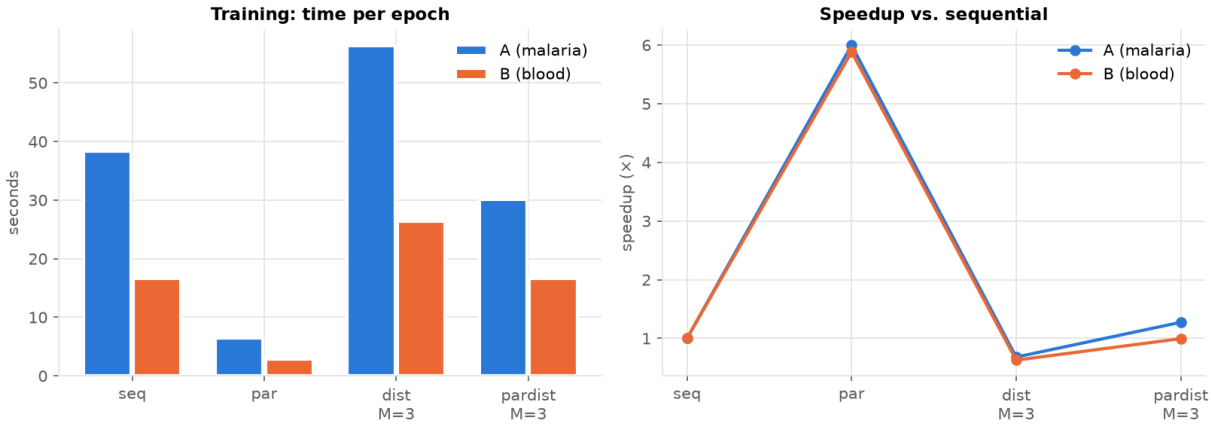



Figure 8: Time per epoch and training speedup.

On a single machine, parallel training is 6 times faster. Distributed over the internet, however, it is **slower than sequential**. In each round of 10 steps, every laptop receives and returns the full model (about 270 KB) through the server in Santiago, and that communication cost outweighs what is gained by splitting the computation. Parallel distributed training partly compensates for it, because each node computes its round with 12 threads. For reference, in an earlier test without a network (3 containers on a single laptop),

distributed $3 \times 1$ did speed up by 3.4× and parallel distributed by 5.7×. Accuracy drops to 80–83 % because each node sees one third of the data between averages and there are only 3 epochs.

## 4.3 Inference in the 4 configurations

| Configuration | Nodes × cores | Throughput | Mean latency | Max. queue A / B | Errors |
|---|---|---|---|---|---|
| Sequential | $1 \times 1$ | 33.2 tx/s | 12 839 ms | 418 / 380 | 0 |
| Parallel | $1 \times 12$ | 52.5 tx/s | 1 412 ms | 0 / 0 | 0 |
| Distributed | $3 \times 1$ | 43.5 tx/s | 1 699 ms | 1 / 3 | 0 |
| Parallel distributed | $3 \times 12$ | 54.1 tx/s | 1 367 ms | 0 / 0 | 0 |

Table 12: Inference: 2000 queries per configuration (1000 from desktop and 1000 from web). The fifth column is the maximum number of messages waiting in `cola_modeloA` and `cola_modeloB`.

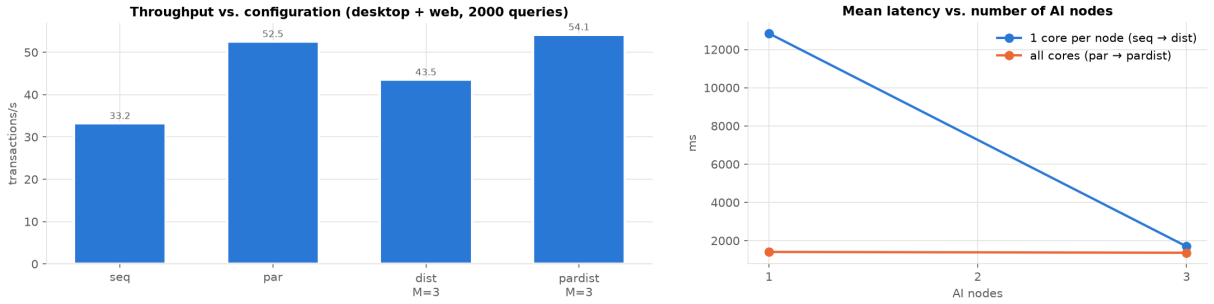



Figure 9: Throughput vs. configuration (left) and latency vs. number of AI nodes (right).

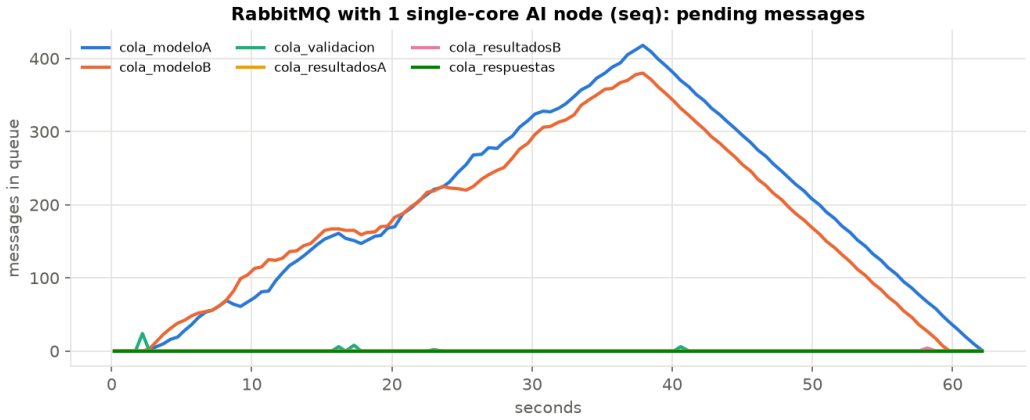



Figure 10: RabbitMQ queues with a single 1-core AI node: the model messages pile up.

In sequential mode, `cola_modeloA/B` reaches about 400 waiting messages and latency rises to 12.8 s (Figure 10). However, the AI node uses only about 20 % of its core: with 1 thread, RabbitMQ delivers one message at a time (*prefetch* 1), and each message waits for a round trip to Santiago before the next one. With 12 threads or with 3 nodes, several messages travel at once, the model queues stay empty and latency drops almost 10 times. From there on the limit is no longer the AI: what waits the most is `cola_validacion` (up to 48 messages), because BD2 validates each query twice from Member01's home. That is why throughput levels off at 52–54 transactions per second.

## 4.4 Saturation test

| Client | Transactions | OK | Errors | Mean latency | p95 | Throughput |
|---|---|---|---|---|---|---|
| Desktop | 1 000 | 1 000 | 0 % | 2 107 ms | 3 498 ms | 16.7 tx/s |
| Web | 1 000 | 1 000 | 0 % | 2 116 ms | 3 449 ms | 16.6 tx/s |
| Android | 1 000 | 1 000 | 0 % | 2 590 ms | – | 32.0 tx/s |

Table 13: Saturation with the three clients at the same time and the 3 AI nodes.

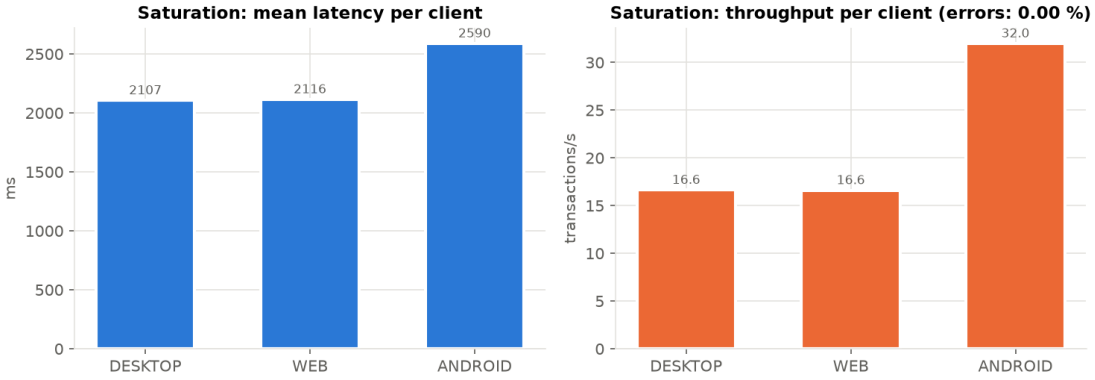



Figure 11: Latency and throughput per client during saturation.

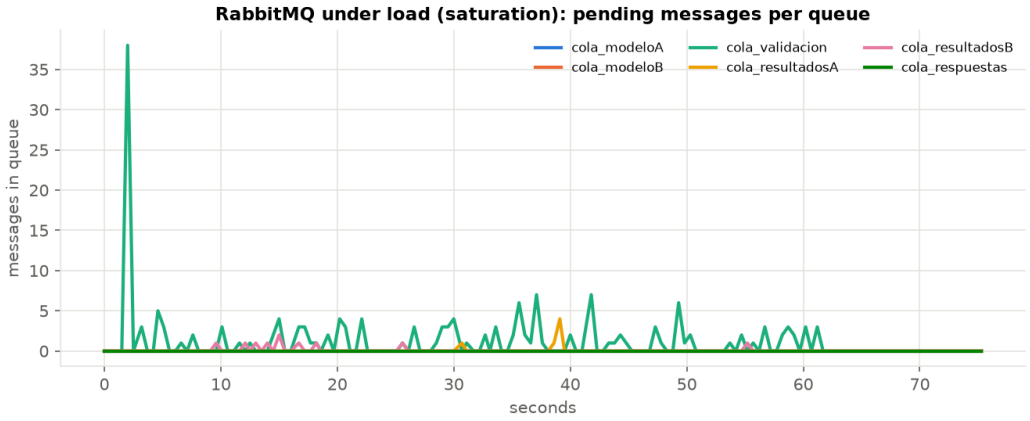



Figure 12: RabbitMQ under load: pending messages per queue during saturation.

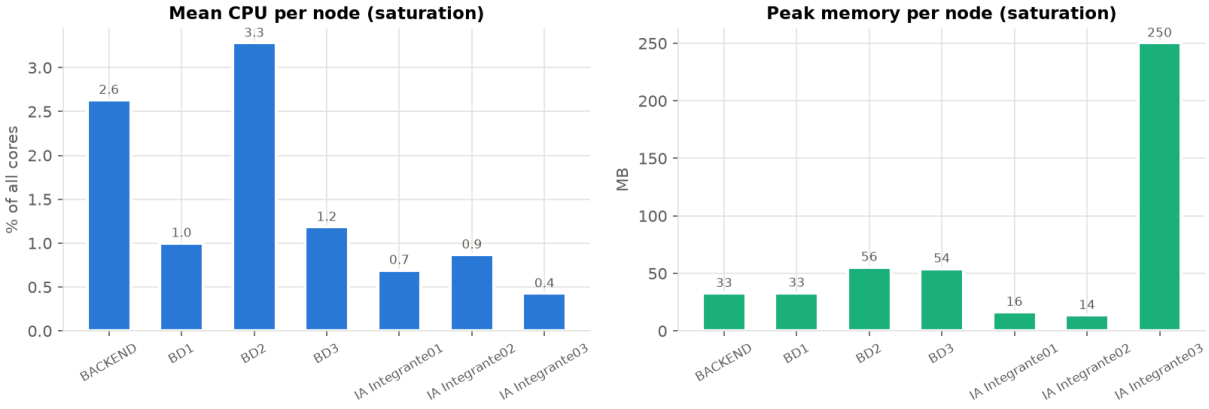



Figure 13: CPU (as a percentage of all the cores of each machine) and memory of each node during saturation.

All 3000 transactions were answered without errors. With the 3 AI nodes, the model queues do not grow; what fluctuates is `cola_validacion` (at most 38 messages), which empties as soon as the burst drops. No node exceeds 10 % CPU: the system is limited by network latency, not by computation. The Android values are the ones the app showed at the end (Figure 2); its CSV was overwritten by a later run without a connection, so there is no p95. A first attempt from another phone ended with 0/1000 because the app had lost its connection before starting; those queries never left the phone.

### 4.5 1000 registrations and database validation

The load generator registered 1000 new random clients: 1000 OK, 116 ms mean latency and 54 registrations per second. At the end, `verificar_consistencia.py` cross-checked the three databases through RabbitMQ:

| **Check** | **Value** |
|---|---|
| Results in BD1 (PostgreSQL, Member02) / BD3 (MySQL, Member03) | 5 835 / 6 473 |
| Valid image validations in BD2 (MariaDB, Member01) | 12 308 |
| Results without validation in BD2 / in both BD1 and BD3 / duplicates | 0 / 0 / 0 |
| Queries reported OK to the client without a stored result | 0 |
| Clients in BD2 (1 004 initial + 1 000 registered) | 2 004 |
| **Result** | **consistent** |

Table 14: Cross-validation of BD1, BD2 and BD3 at the end of the tests.

### 4.6 General discussion

The results show that parallelism works on a single machine, both for training (6 times faster) and for serving queries (almost 10 times lower latency). Distribution over the internet, on the other hand, does not speed up training: the cost of sending the model in every round exceeds the cost of the computation. This is the parallelization cost studied in the course; to take advantage of distribution, it would be advisable to synchronize less often or to have the nodes on a local network. In inference, RabbitMQ shows where the bottleneck is at each moment: first in the AI nodes and, as threads and nodes are added, in the BD2 validation.

The limits of the measurements should also be considered: one run was made per configuration, over variable home networks, with laptops with different processors, and the load was launched from a laptop that was also running BD1 and an AI node. The figures describe the behavior of the complete system rather than a *microbenchmark* under identical conditions.

## 5 Conclusions

- A distributed system was implemented with Android, desktop and web clients, a backend with custom sockets, RabbitMQ as the only *middleware*, AI nodes in Go and three heterogeneous databases. It

worked end to end with the server on Google Cloud and three laptops in three homes.
- The CNNs written from scratch, with convolution and MLP *micro chunks*, reach 95.2 % on malaria and 88.6 % on 8 types of blood cells.
- On a single machine, parallel training is 6 times faster. Distributed over the internet it is slower than sequential, because communication weighs more than computation; without a network, the same scheme speeds up 3.4 times with 3 nodes.
- In inference, going from 1 thread to several threads or nodes lowers latency almost 10 times; after that, the bottleneck moves to the BD2 validation.
- The system answered 11 000 transactions without errors, including the saturation with the three clients at the same time. Transactions, `FOR UPDATE`, idempotent inserts and one connection per thread kept the three databases consistent.

# 6 Source code appendix

The complete source code is included in the ZIP file submitted with this report and in the repository `https://github.com/dodo-UNI-2026-02/BorgAICube`. The main files are:

- `codigo/java/src/cubo/`: backend, web server, desktop clients and load generator.
- `codigo/go/`: CNN, AI node and training in the 4 configurations.
- `codigo/python/`: DB nodes, validator, monitor, consistency check and charts.
- `codigo/android/`: Android app.
- `codigo/web/`: web chat.
- `codigo/sql/`: creation scripts for BD1, BD2 and BD3.

# 7 Documentation appendix

## Requirements

- Java 17 or later, Go 1.24 or later and Python 3.10 or later (with `pika`, `psycopg2` and `pymysql`).
- RabbitMQ, and on each DB node its engine: PostgreSQL, MariaDB or MySQL.
- Android Studio, only to build the app.

## Server (on Google Cloud)

```
curl -fsSL https://raw.githubusercontent.com/dodo-UNI-2026-02/BorgAICube/main/scripts/
    nube_preparar_vm.sh -o preparar.sh && bash preparar.sh
bash ~/BorgAICube/scripts/servidor_nube.sh
```

## Nodes (on each laptop)

```
$env:CUBO_RABBIT_URL = "amqp://cubo:PASSWORD@34.176.215.27:5672/cubo"
python codigo\python\nodo_bd.py --rol BD2 --db-host 127.0.0.1     (BD1 or BD3 depending on
    the laptop)
.\nodo.exe -datos datos
```

## Clients

```
java -cp "codigo/java/out;codigo/java/lib/*" cubo.desktop.ClienteDesktop --host
    34.176.215.27
http://34.176.215.27:8080
```

## Measurements

```
./scripts/equipo.sh seq | par | entrenar_local | pardist | entrenar_dist | saturacion |
    registros | dist | validar
python codigo/python/graficas.py --resultados resultados/equipo [--en]
```

The results are stored as CSV files in the `resultados/equipo/` folder, and the charts as PNG files in its `graficas/` subfolder (`graficas_en/` for the English version).

# References


[1] CC4P1 – Programación Concurrente y Distribuida. *Práctica 03 2026-II: Sistema distribuido de CNN con chat multiplataforma y middleware RabbitMQ*. Universidad Nacional de Ingeniería.

[2] S. Rajaraman et al. *Pre-trained convolutional neural networks as feature extractors toward improved malaria parasite detection in thin blood smear images*. PeerJ, 6:e4568, 2018. Dataset: National Library of Medicine (NIH).

[3] J. Yang et al. *MedMNIST v2: A large-scale lightweight benchmark for 2D and 3D biomedical image classification*. Scientific Data, 10:41, 2023.

[4] A. Acevedo et al. *A dataset of microscopic peripheral blood cell images for development of automatic recognition systems*. Data in Brief, 30:105474, 2020.

[5] RabbitMQ. *RabbitMQ documentation: queues, consumer prefetch and RPC*. `https://www.rabbitmq.com/docs`.

[6] CC4P1 course material. *Desafío de los Sistemas Distribuidos*.

[7] CC4P1 course material. *Monitores, condición de carrera y thread-safe (Hilos_v12)*.

[8] CC4P1 course material. *Desempeño y optimización para la latencia (Desempeño251_v03)*.

[9] Google Cloud. *Compute Engine documentation*. `https://cloud.google.com/compute/docs`.